

Faster Activation Functions at the Edge for Post-Training Speedups

Anton Lydike¹ Jun Bi² Jackson Woodruff¹

Abstract

On-device AI has gained significant attention for enabling efficient, low-latency inference on edge devices. However, tight resource constraints on these platforms make the deployment of accurate and lightweight deep learning models challenging. In particular, advanced activation functions (AFs) like Swish and GELU often incur high inference overhead due to the lack of hardware fast-paths for exponentiation and division, restricting edge-ML applications to simple AFs like ReLU, limiting model accuracy. To address this, we propose FFCC, a compiler that automatically generates efficient approximations of AFs through floating-point reinterpretation. These functions do not require hardware fast-paths, meaning they remain fast on edge devices, but are accurate enough to be used as post-training drop-ins. FFCC takes a specification of AFs using basic floating-point operators and applies derivation rules to lower these expressions into efficient instruction sequences. Our experiments show that FFCC provides fast approximations of AFs, achieving order-of-magnitude speed-ups over accurate baselines on Arm M7, Aarch64 and Intel platforms. Using ConvNeXt as an example, we demonstrate how these activation-level gains translate to end-to-end speed-ups, and do not result in significant loss of model accuracy.

1. Introduction

On-device AI has emerged as a promising paradigm for efficient, low-latency, low-costs and privacy-preserving inference (Moon et al., 2024; Wang et al., 2025). By executing models locally, on-device AI reduces network de-

¹Institute for Computing Systems Architecture, The University of Edinburgh, Edinburgh, United Kingdom ²State Key Lab of Processors, Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China. Correspondence to: Anton Lydike <anton.lydike@ed.ac.uk>.

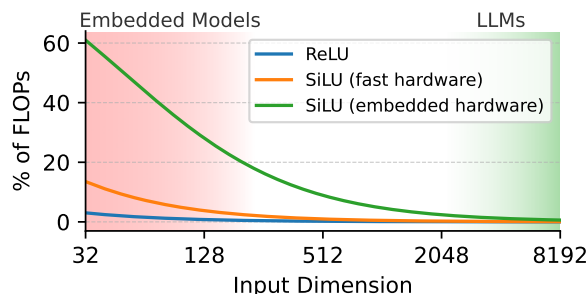


Figure 1. Fraction of per-layer flops dedicated to activations as a function of input-dimension. Models that fit on embedded devices (red region) also suffer from missing fast-paths in hardware. This seems counter-intuitive as overhead quickly diminishes for large models (green region).

pendence and avoids transmitting sensitive data to remote servers. It has been widely adopted in Internet-of-Things applications (Menshchikov et al., 2021; Li et al., 2018), autonomous driving (Liang et al., 2022; Gong et al., 2023), and medical devices (Tuli et al., 2020; Mock et al., 2023). However, the stringent compute and memory constraints of edge devices require models to be lightweight, with substantially reduced parameter counts and FLOPs.

The academic community has explored a wide range of activation function (AF) designs in order to solve problems with learning instability around the zero point commonly observed in ReLU. For example, GELU (Hendrycks & Gimpel, 2023) and SiLU (Elfwing et al., 2018) have been widely adopted in Transformer-based architectures (Radford et al., 2023; Dosovitskiy, 2020; Liu et al., 2021; Zhao et al., 2023), while in convolutional neural networks, functions such as Swish (Ramachandran et al., 2017) and SMU (Biswas et al., 2021) have shown notable improvements in accuracy, especially for lightweight models (Seo et al., 2024). These advanced activation functions are efficient for training and inference on modern accelerators (e.g., GPUs).

However, when deployed to resource-constrained devices, such AFs often introduce non-negligible inference overhead due to the lack of dedicated hardware fast-paths, for e.g. exponentiation. As illustrated in Figure 1, the relative computational cost of activation functions becomes increasingly significant as the total model size decreases. In particular,

Table 1. Commonly used activation functions and their characteristics

Name	Expression	Typical Usage	Expensive Operations
ReLU	$\text{ReLU}(x) = \max(0, x)$	CNNs	-
ReLU6	$\text{ReLU6}(x) = \min(6, \max(0, x))$	CNNs	-
SiLU / Swish	$\text{SiLU}(x) = \frac{x}{1+e^{-x}}$	EfficientNet, YOLO	div, exp
GELU	$\text{GELU}(x) = 0.5x \cdot (1 + \text{erf}(x2^{-0.5}))$	Transformers	div, erf (tanh, exp)
Softmax	$\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$	Transformers	exp
Softplus	$\text{Softplus}(x) = \log(1 + e^x)$	VAE models	log, exp

SiLU incurs much higher latency compared to ReLU, with the gap widening in ultra-lightweight networks, where the activation overhead can reach or exceed 50% for small sizes (input dimensions of 64 and smaller). Such overhead makes application of advanced AFs impractical for on-device models, leading to a dilemma between accuracy and efficiency.

Both hardware (Reggiani et al., 2025; Nilsson et al., 2014; Meher, 2010) and software (Schraudolph, 1999; Perini & Reitz, 2018; Moroz et al., 2022; Wulfert et al., 2024) approximations have been proposed for reducing the activation overhead without sacrificing accuracy. However, these approaches either require specialized hardware support or a manually designed effort tailored to specific AFs. For example, Flex-SFU (Reggiani et al., 2025) leads to an increase of 5.9% in chip area for hardware support. This lack of generality limits their applicability, thus hindering the exploration of novel AF designs for on-device models, and requires entirely new hardware, which comes with additional costs.

To address this challenge, we propose FFCC (Fast Floating-point Compiler), a hardware-aware compiler that automatically generates efficient approximations for a wide range of activation functions, without requiring special hardware. FFCC introduces a domain-specific language (DSL) that allows users to express activation functions using a set of basic operators. It then applies symbolic derivation rules to lower the computation graph into efficient instruction sequences, leveraging floating-point bit reinterpretation for efficient yet accurate approximation. To further minimize approximation error, FFCC optimizes constants used in the approximation through a search-based tuning process. By performing approximation generation and optimization, FFCC eliminates the need for manual designs or specialized hardware.

We evaluate the performance of FFCC on three different activation functions, performing accuracy tests which show that our activation functions are an order of magnitude more accurate than existing software approximations like h-swish. We further explore the speedup that our activation functions achieve, showing 13 – 26 $\times$ improvement in speed over exact alternatives on embedded and mobile hardware. Finally, we look at ConvNeXt as a case study, evaluating end-to-

end accuracy to show that FFCC’s GELU approximation is accurate enough to be used as a drop-in post-training replacement and provides a measurable performance increase.

We make the following contributions: (1) A tool for automatically applying cast-based optimizations to generate fast-but-accurate approximations, which we use to (2) provide fast approximations for the commonly used swish/SiLU, Tanh, and GeLU activation functions. Further, we show that (3) These functions are accurate enough to be used as a post-training drop-in replacement.

2. Background and Motivation

There have been extensive improvements made to machine learning activation functions over the last 10 years. However, not all models can make use of these advances due to the prohibitively expensive computations required to calculate some of these functions. Activation functions play a crucial role in deep learning models by introducing non-linearity into the network, enabling it to learn complex patterns and representations from data, and have been exhaustively studied. Table 1 summarizes the mathematical definitions of the most widely used activation functions.

In 2015, ReLU was by far the most widely-used activation function (Andri et al., 2025) because it is computationally efficient — but research has moved on because it suffers from problems like dying neurons. Seo et al. (2024) demonstrates that replacing ReLU with smoother alternatives generally results in increased model accuracy. Large parts of machine learning have since switched away from ReLU, with GeLU and SiLU being the most popular activation functions for transformer-based architectures today (Andri et al., 2025) and softmax features in attention mechanisms.

However, modern activation functions involve computing results of expensive functions such as exponentiation, logarithm and even the error function. These operations lack native hardware support on edge devices, leading to non-negligible inference overhead when used. Only in these extremely compute-constrained environments have we seen ReLU persist as an activation function, as the better alterna-

tives do not perform well enough.

Various works have been done to find suitable replacements that perform well on-edge hardware. The most well-known approach is Hardswish (or h-swish) (Howard et al., 2019) as part of MobileNet v3, which uses $\text{ReLU } 6(x)/3 - 0.5$ to very roughly approximate $\sigma(x)$. In this paper, we will present an alternative to h-swish that makes use of the bit-level representations of floating-point numbers to achieve a more accurate approximation that is as fast as h-swish.

3. Approximations Based On Bit-Representations

At the heart of FFCC are two approximations that provide piecewise-linear approximations of $\log_2 x$ and 2^x with just one multiply-accumulate and one bit-casting operation. To understand how we derive these approximations, we will introduce the IEEE floating-point format, formalize what it means to perform a bit-cast from such a floating-point format to an integer, how these operations can be used to generate approximations, and finally, how such approximations can be tuned to minimize error.

3.1. Floating Point Representation

Floating-point numbers are typically represented using the IEEE 754 standard, which encodes a number using three components, including a sign bit (s), an exponent (k), and a mantissa (m). These three (integer) values are combined to produce a floating-point number according to this formula:

$$\text{fp}(s, k, m) = -1^s 2^{k-B} (1 + m/L) \quad (1)$$

Here B is the exponent bias and L is the scaling factor for the mantissa, which are both dependent on the bit-width of the floating-point type.

For any floating-point number x , the three values (s, k, m) can be calculated with the help of a helper value e (signifying the raw value of the exponent) as follows:

$$\begin{aligned} e(x) &= \lfloor \log_2 x \rfloor \\ s(x) &= \begin{cases} 1 & \text{for } x < 0 \\ 0 & \text{otherwise} \end{cases} \\ k(x) &= e(x) + B \\ m(x) &= \frac{x - 2^{e(x)}}{2^{e(x)}} L \end{aligned} \quad (2)$$

To shed some light on the value of m , it can be thought of as a way to interpolate between different exponents. When $m = 0$ the value of the float is a whole power-of-two ($-1^s 2^{k-B}$), as m approaches L , the result approaches

$$\begin{aligned} &(-1)^s \cdot 2^{e-B} \cdot (1 + m/L) \\ &\text{0 1 0 0 0 0 0 1 0 0 0 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0} \\ &(-1)^0 \cdot 2^{132-B} \cdot (1 + 0.375) = 44 \end{aligned}$$

Figure 2. Encoding of the number 44 as an IEEE 32-bit floating-point number, separated into sign, exponent, and mantissa parts.

the next power-of-two: $-1^s 2^{e+1-B}$, and $m = 0.5L$ sits half-way in between these two values.

3.2. Casting to Integers

The 32-bit single-precision format (float32) allocates 1 bit for the sign, 8 bits for the exponent (with B equals to 127), and 23 bits for the mantissa (with L equals to 2^{23}). These fields are encoded in memory in the order (s, k, m) , resulting in the highest bit as the sign-bit, followed by 8 bits of biased exponent, and 23 bits of mantissa (See Figure 2). The final bit-pattern for any 32-bit float x can therefore be described as the following integer consisting of the three parts (s, k, m) :

$$\begin{aligned} \text{int}_{32}(x_{fp}) &= s(x)2^{31} + k(x)2^{23} + m(x) \\ &= s(x)2^{31} + (\lfloor \log_2 x \rfloor + B)L + \frac{x - 2^{e(x)}}{2^{e(x)}} L \\ &= s(x)2^{31} + (\lfloor \log_2 x \rfloor + B + \frac{x - 2^{e(x)}}{2^{e(x)}})L \end{aligned} \quad (3)$$

Notably, for positive values (when $s = 0$), the integer-representation of a floating-point number is simply $(\lfloor \log_2 x \rfloor + B + \frac{x - 2^e}{2^e})L$. Dividing by constant L and subtracting B , we are left with $\lfloor \log_2 x \rfloor + \frac{x - 2^e}{2^e}$.

We can see that the first part is an integer approximation of $\log_2 x$, while the second part interpolates between values (as m goes $0..L$, m/L goes $0..1$). We can hence formulate that the integer representation of a floating-point value approximates its $\log_2$.

$$\begin{aligned} \text{int}(x) &\approx (\log_2 x + B)L \\ \log_2 x &\approx \frac{\text{int}(x)}{L} - B \end{aligned} \quad (4)$$

We can now understand the positive-value restriction, as $\log_2$ is defined for positive values only. Hence, this approximation is only sensible for positive floating-point values.

We will call this operation of changing the interpretation of the underlying bits *bit-casting*.

3.3. Casting to Float

From here, we can quickly convince ourselves that the floating-point representation of an integer value approximates exponentiation with base 2. One way is to consider that bit-casting from float to int and back to float will not change the underlying value of the bits. Since the first operation – re-interpreting a float as an integer – provides an approximation of $\log_2$, the bit-cast back must be the inverse of this operation. We can come to a similar conclusion when rewriting Equation 4.

Casting to float provides the following approximation:

$$2^x \approx \text{fp}(L(B + x_{\text{int}})) \quad (5)$$

Where x_{int} reminds us that x is an integer value.

3.4. Derivatives

Seeing as we are attempting to use these approximations in the framework of machine learning, having a more formal specification of these operators and their derivatives might prove useful. For this, we focus on the “happy path” of positive, normal floating-point numbers, as this keeps the formulae much neater. Extending this to the full range of floating-point numbers is outside the scope of this work.

To find the derivatives, we rewrite Equation 3 into a union of affine functions of the form $c_1x + c_2$. Each segment corresponds to one exponent value, hence this union is over the set of E of possible exponents (in the 32-bit case $E = \{-B, \dots, 2^8 - B - 1\}$):

$$\begin{aligned} \text{fp}(x) &= \bigcup_{k \in E} \left\{ 2^{k-B} \left(\frac{x}{L} - k + 1 \right) \mid x \in [Lk, L(k+1)) \right\} \\ \text{int}(x) &= \bigcup_{k \in E} \left\{ L \left(\frac{x}{2^k} + k + B - 1 \right) \mid x \in [2^k, 2^{k+1}) \right\} \end{aligned} \quad (6)$$

The derivatives of these piecewise-affine functions are then these piecewise-constant functions:

$$\begin{aligned} \text{fp}'(x) &= \bigcup_{k \in E} \left\{ \frac{2^{k-B}}{L} \mid x \in [Lk, L(k+1)) \right\} \\ \text{int}'(x) &= \bigcup_{k \in E} \left\{ \frac{L}{2^k} \mid x \in [2^k, 2^{k+1}) \right\} \end{aligned} \quad (7)$$

For the 32-bit case, $L = 2^{23}$ and $B = 127$, and hence the derivatives are further simplified to 2^{k-150} and 2^{23-k} respectively. Plugging these definitions into PyTorch then allows us to leverage Torch’s autograd for various tasks.

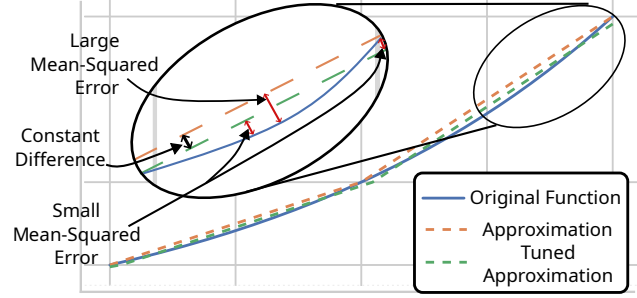


Figure 3. Subtracting a small $\varepsilon \approx 0.045$ from x increases approximation accuracy.

3.5. Accuracy

While the approximations of Equation 4 and Equation 5 are correct, they consistently under-/ and over-approximate (for the $\log_2$ and 2^x case, respectively). We can reduce the error significantly by offsetting the curve by a small amount, simply by changing the bias value B slightly. In the exponentiation approximation, subtracting 0.045 from B , the error is roughly halved (see Figure 3). A better approximation could hence be formulated as $2^x \approx \text{fp}(L(B - 0.045 + x_{\text{int}}))$.

The process of picking the “best” constant is referred to as *constant tuning*, and usually aims to minimize *maximum relative error*. Notably, (Kadlec, 2010) found that tuning this constant in isolation does not necessarily yield the lowest error when the approximation is used as part of a larger expression. Kadlec found that a holistic optimization of all constants in the final program is able to reduce maximum relative error by $2.7\times$ over the approach of just looking at the bitcast. His final constants are found by exhaustively searching all values around previously found candidates.

4. Implementation

We present FFCC, a compiler to compile exact activation functions into approximations, leveraging approximations introduced in section 3. FFCC is written in Python and has a Static-Single-Assignment (SSA) based intermediate representation (IR) with a simple greedy rewriting engine, similar to other DSLs (Fehr et al., 2025; Bisbas et al., 2024; Van Delm et al., 2026).

FFCC’s pipeline is split into stages as outlined in Figure 4, where the code is first normalized, then approximations are applied, then magic constants are tuned for accuracy, and finally the code is converted into the needed target representation (PyTorch, Numpy, LLVM).

In addition to publishing FFCC as an open-source tool at github.com/AntonLydike/ffcc, we provide a detailed breakdown of each step in the following sections.

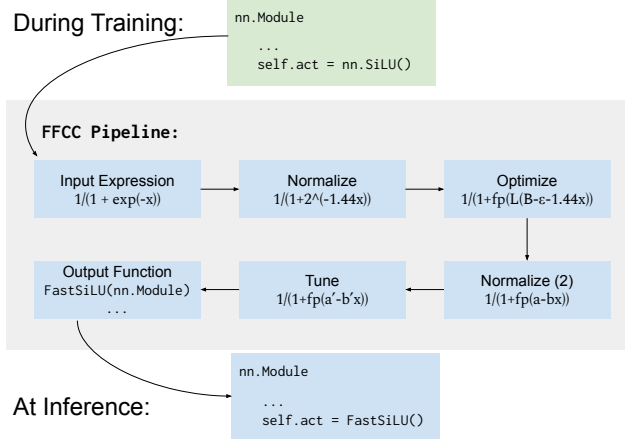


Figure 4. FFCC facilitates generation of fast approximations of activation functions for use at inference time on embedded devices through a simple pipeline consisting of small individual pattern rewrites and a constant-tuning phase.

4.1. Normalization and Constant Folding

The provided expression might not immediately make a good candidate for approximation. We therefore rewrite the expression into a more suitable form to expose the most “surface area” for potential approximations. FFCC applies a number of reduction rules to bring the expression into a canonical form. Both simple rewrites, such as $A + -x$ to $A - x$, as well as more complex operations, such as cancelling factors in divisions, are applied to the intermediate representation.

These rules are much more aggressive than the rules of traditional compilers. As we do not maintain perfect accuracy, we can perform aggressive rewrites that influence final program accuracy. The most important normalizations ensure that logarithms and exponentiations are in base-2 form, which we achieve with the following identities:

$$\log_b(x) = \frac{\log_2(x)}{\log_2(b)} \quad (8)$$

$$b^x = 2^{\log_2(b)x}$$

We also perform constant-folding as part of our simplification. For example, after rewriting $\ln(x) \mapsto \frac{\log_2(x)}{\log_2(e)}$, we can calculate $c = 1/\log_2(e) = 0.693\dots$ and simplify the expression to $c\log_2(x)$.

The result of the normalization pass is a heavily simplified intermediate representation that makes approximation opportunities clear.

Table 2. Rewrite rules based on Equation 5 and Equation 4.

Match	Replace With	Condition
2^A	$\text{fp}(L(B - \varepsilon + A))$	A input-dependent
$\log_2(A)$	$\frac{\text{int}(A)}{L} - B + \varepsilon$	A input-dependent

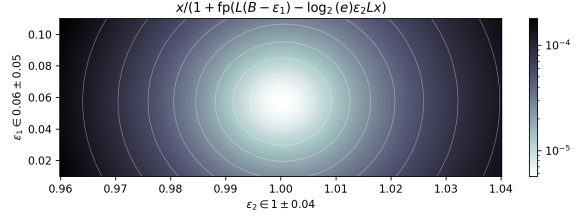


Figure 5. Adjusting the two constants present in this approximation of SiLU and plotting the resulting mean-squared-error provides a smooth surface for gradient-descent-based optimizations.

4.2. Applying Approximations

At this stage, the intermediate representation (IR) contains base-2 logarithms and exponentiations. The compiler now replaces instances of these with our approximations based on simple peephole-rewrites (see Table 2). Before applying approximations, we ensure that the inner term A is input-dependent. For increased accuracy (See subsection 3.5), small constant offsets (ε) are present in the inserted code. These values are marked as “tunable” in the IR.

After the approximations are inserted, another round of normalization and constant folding follows. The constants introduced during the approximation step can often be merged with surrounding values. This constant-folding process keeps track of “tunable” attributes on constants and preserves them throughout. These annotations are then used in the next step to fine-tune constants for maximum accuracy.

4.3. Constant Optimization

The approximations we generate contain annotations marking the location of tunable constants. To optimize these parameters, we use an approach specifically aimed at machine-learning applications that minimizes the mean-squared error of an approximation around zero. We use PyTorch to perform gradient descent on the parameters to find an optimal solution to this problem, relying on the derivatives from subsection 3.4 to guide the gradient descent through the optimization space. The loss landscape of a SiLU-approximation containing two tunable parameters is shown in Figure 5. This process is quick to perform, as the start values for these constants are close to optimal.

4.4. Lowering

The final step is to convert the FFCC IR to externally usable code. For this, we provide multiple output formats. FFCC can generate either PyTorch Modules (Ansel et al., 2024) or Numpy expressions of the approximated function. FFCC also provides an additional direct-to-LLVM conversion that generates LLVM-IR (Lattner & Adve, 2004).

5. Evaluation

We demonstrate that our approximations enable edge-deployment of more complex activations by achieving order-of-magnitude speedups over exact implementations, which translate into end-to-end gains for real-world architectures without sacrificing significant model accuracy.

Activation Micro-Benchmarks: We establish the relative performance of ReLU to more advanced SiLU, GELU, and tanh activation functions on three architectures, demonstrating significant speedups over SOTA implementations.

End-to-End Analysis: We provide an in-depth performance breakdown of the ConvNeXt architecture, showing per-stage speedups gained by replacing its GELU with our functions.

Model Accuracy Assessment: We characterise the accuracy of our approximations and show that they preserve model accuracy in most cases, with a discussion of the limitations and possible mitigations.

5.1. Our Approximations

Our experiments use three different activations, SiLU, GELU and tanh, to showcase the ability of FFCC to generalise past a single activation. FFCC automatically generated PyTorch modules for these functions, as shown in Listing 1. For the GELU implementation we use the approximation $\text{GELU}(x) = x\sigma(1.702x)$, introduced in the original paper (Hendrycks & Gimpel, 2023).

5.2. Edge Performance Microbenchmarks

We demonstrate that our approximations represent a significant shift in the performance of activation functions at the edge, cutting overhead of activations such as SiLU, GELU or tanh to just 15 – 16% of their original cost on low-power edge devices, and by up to 26 $\times$ on higher-power mobile chips. Our benchmarks compare activations on three different platforms: an Arm Cortex-M7 based STM Nucleo dev-board (STM32F722ZE MCU), a Snapdragon 8 Elite Gen 4, and a laptop-class Intel i7-1260p. All experiments were compiled with clang using `-O3 -ffast-math` and appropriate `march` flags.

Our cycle-accurate measurements on the Arm M7 platform demonstrate that our approximations are able to cut the

Listing 1 PyTorch modules generated with FFCC.

```
from torch import nn, int32, float32

class FastSiLU(nn.Module):
    def forward(self, x):
        v0 = (x * -12104087.0 + 1064873152.0)
        v1 = v0.type(int32).view(float32)
        return x / (1 + v1)

class FastGELU(nn.Module):
    def forward(self, x):
        v0 = (x * -21513938.0 + 1064888768.0)
        v1 = v0.type(int32).view(float32)
        return x / (1 + v1)

class FastTanh(nn.Module):
    def forward(self, x):
        v0 = (x * 24192124.0 + 1064872640.0)
        v1 = v0.type(int32).view(float32)
```

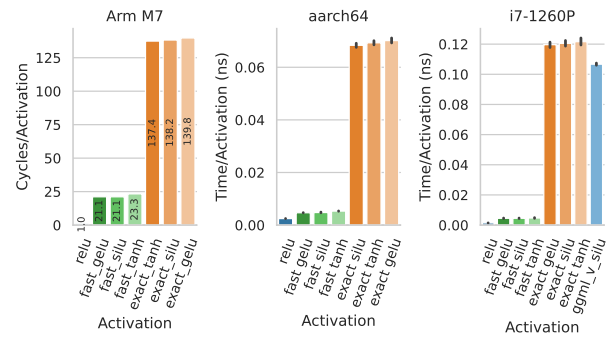


Figure 6. Micro-Benchmarks confirm large speedups of our approximations (green) over exact baselines (orange) on various architectures, moving performance closer to embedded activations such as ReLU. Additionally, we significantly outperform another SOTA implementation of SiLU taken from llama.cpp (blue).

overhead associated with exact implementations from about 130 cycles down to less than 24, a reduction to just 15 – 16% of the original cycle counts. On the Snapdragon and Intel chips, our speedups in wall time are even larger (15 – 22 $\times$), where we even outperform a hand-tuned inline-assembly implementation of SiLU that is shipped in ggml (llama.cpp) by 23 $\times$. Our large speedups on Intel are partly due to the nature of the shared integer/floating-point register file of AVX2, making bit-casting operations a no-op (Figure 6).

5.3. End-to-End ConvNeXt Experiments

In order to demonstrate that these activation-level speedups translate to the model-level, we perform an in-depth evaluation of the ConvNeXt (Liu et al., 2022) model families. The ConvNeXt architecture is centered around blocks of depth-wise and point-wise convolutions. These blocks are organised into four stages, with spatial resolution decreasing and channel count increasing between stages. Each block

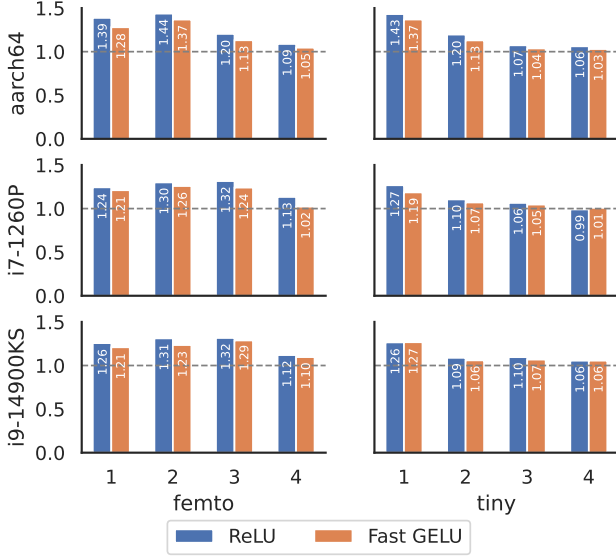


Figure 7. When replacing the exact GELU in ConvNeXt with our approximation, we see speedups across architectures (orange) in most stages of the ConvNeXt architecture. Replacing GELU with ReLU (blue) shows that our approximations almost reach performance parity with the fastest edge activation.

consists of a depth-wise convolution, normalisation, point-wise convolution (expansion), GELU activation, another point-wise convolution (squeezing) and residuals. This is representative of typical SOTA model architectures, where activations are a relatively small fraction of total FLOPs.

We perform an in-depth analysis of the per-stage activation overhead of smaller ConvNeXt models (the Femto and Tiny classes) on aarch64 (Snapdragon 8 Elite Gen 4) and two intel-based CPUs. In our experiment we replace the exact GELU of ConvNeXt with our approximation to gauge the speedups unlocked by our activation. We also evaluate with ReLU to obtain data for peak achievable speedup.

For the Femto model, we demonstrate speedups of around $1.2\times$ to $1.37\times$ concentrated around stages 1-3 with no changes to the last stage. The larger “Tiny” model reduces overall speedups as expected, as a larger percentage of flops is spent on the convolutions. Here we only gain significant speedups in Stage 1, where we observe around $1.19\times$ to $1.37\times$ speedups (Figure 7)

When comparing the speedups gained by ReLU to what is already achieved by our approximations, we can see that we capture most of the available headroom available by switching activation functions (Figure 7). While switching to ReLU would result in additional speedups, it comes with significant drawbacks, such as requiring complete model-retraining, which our activations avoid.

5.4. Approximation Error

We show that the approximations we generate are generally able to replicate the broader structure of the activation functions to a much higher degree than previous work (e.g., Hardswish). Our generated approximations demonstrate a high degree of accuracy compared to using functions like Hardswish to approximate SiLU.

Figure 8a shows the accuracy of our approximations (orange dots) overlaid on the original functions (in gray). We have also plotted hardswish (which can be seen as an approximation of SiLU) and ReLU for comparison. Figure 8b shows a log-plot of the actual function errors, where we can see that FFCC’s generated approximations are nearly always within 0.01 of the true function, compared to approximations like h-swish, which are frequently between 0.01 and 0.1 in absolute error.

To fine-tune the constants, we used PyTorch to perform gradient descent as outlined in subsection 4.3, using Adam optimizer with an initial learning rate of $1e6$, with StepLR $\gamma = 0.1$ and a step size of 400, training for 1000 epochs. Using MSE Loss over $1e6$ samples between -6 and 6 as our criterion. We achieved final loss of $5.67e-6$ for SiLU, $6.1e-6$ for GeLU and $8.82e-6$ for tanh.

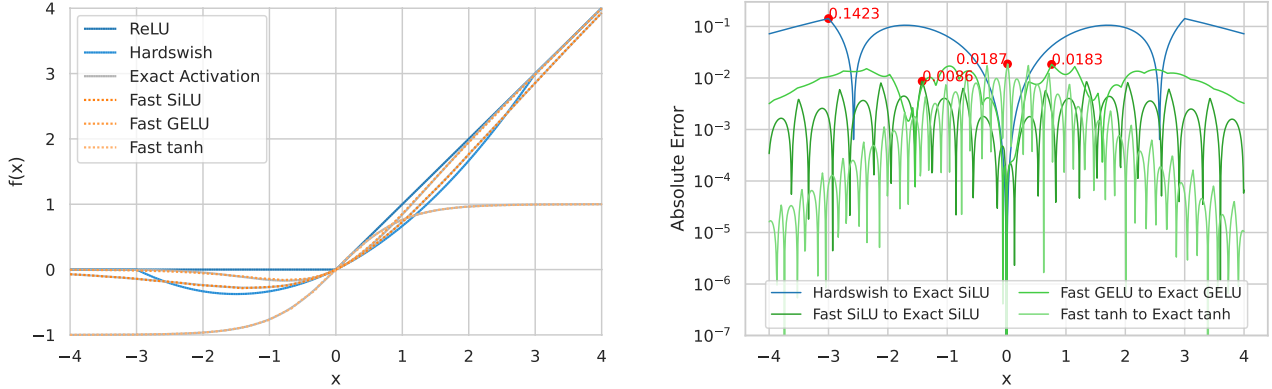
5.5. Post-Training Approximation

We now show that FFCC-generated approximations can be used as *post-training replacements* with minimal losses to model accuracy in most cases. We focus on SiLU as a case study here. For each benchmark, we train one model with Hardswish and one with exact SiLU. We then construct a third model by replacing SiLU with FastSiLU while reusing the same trained weights. Thus, SiLU and FastSiLU differ only in the deployed activation implementation.

All models are trained and evaluated on an NVIDIA A30 GPU following (DeVries & Taylor, 2017): an initial learning rate of 0.1, divided by 5 at epochs 60, 120, and 160, for 200 epochs, with batch size 128, weight decay $5e-4$, and Nesterov momentum 0.9.

We evaluate MobileNetV2 (Sandler et al., 2019), ShuffleNetV2 (Ma et al., 2018), DenseNet-121 (Huang et al., 2018), ResNet-18 (He et al., 2015), and SqueezeNet (Iandola et al., 2016), together with the image classification, keyword spotting, and anomaly detection networks from MLPerf Tiny v1.1 (Banbury et al., 2021). We do not train directly with FastSiLU; instead, we keep the training pipeline unchanged and treat the approximation as an implementation detail of the deployment runtime.

Table 3 reports the results. In 7 out of 8 models, FastSiLU preserves accuracy with no observable drop compared with exact SiLU. The only failure case is the MLPerf Tiny



(a) Plotting our approximations (orange) against exact baselines (gray) shows that they accurately reproduce the original functions.

(b) Absolute error plots showing errors not exceeding 0.019 over the entire domain, almost an order-of-magnitude less than Hardswish.

Figure 8. Accuracy and Error plots for our activations after constant tuning reveal very close matches to the reference implementations.

Anomaly Detection model, an auto-encoder whose activations span a much wider numerical range. In this case, the approximation becomes numerically unstable, similar to post-training quantization failures when activations fall outside the well-approximated range.

This limitation can however be mitigated with range-safe variants. For exp-based activations in single precision, instability occurs outside the numerically safe range of the bit-cast approximation, roughly below -74 and above 74. FFCC could emit guards based on the asymptotic behavior of the target activation. For example, for SiLU, which approaches 0 as $x \rightarrow -\infty$ and x as $x \rightarrow +\infty$, FFCC could generate `if (x < -74) return 0; if (x > 74) return x; return act(x);`. Such guards would add a small overhead while preventing overflow or underflow. We leave systematic range-safe code generation and activation-distribution-aware replacement policies to future work.

Finally, Hardswish performs worse than SiLU and FastSiLU across most benchmarks. While useful when models are trained with it, Hardswish is less suitable as a general post-training replacement for SiLU. In contrast, FastSiLU closely preserves the original activation and can be applied after training with minimal impact on accuracy.

5.6. ConvNeXt Accuracy Deep-Dive

Our accuracy results for ConvNeXt (Liu et al., 2022) below confirm that we maintain model accuracy on modern model architectures too. In our experiment we take a pre-trained ConvNeXt model and show that replacing GELU with our generated approximation yields minor impacts on model accuracy overall. ConvNeXt-Tiny sees a Top-1 accuracy drop of 0.1 percentage points (pp) while Top-5 accuracy remains unchanged. ConvNeXt-Femto sees a drop in accuracy of

Table 3. Comparison of activation functions across different benchmarks. Best results for each benchmark are highlighted in bold.

Model	SiLU	Fast-SiLU	H-Swish
DenseNet-121	74.0	74.0	72.1
MobileNet-V2	69.1	69.5	67.6
ResNet-18	75.4	75.4	74.2
ShuffleNet-V2	70.8	71.1	70.3
SqueezeNet	69.7	69.8	69.6
Image Class.	83.84	83.86	78.24
Kw Spotting	83.36	83.42	82.95
Anomaly Detect.	91.94	<i>nan</i>	90.51

Table 4. ImageNet accuracy of ConvNeXt models before and after replacing exact GELU with the FFCC-generated approximation.

Model	Metric	GELU	FastGELU
ConvNeXt-Tiny	Top-1	82.1	82.0
ConvNeXt-Tiny	Top-5	95.9	95.9
ConvNeXt-Femto	Top-1	77.4	77.2
ConvNeXt-Femto	Top-5	93.6	93.5

0.2pp (Top-1), and 0.1pp (Top-5) (Table 4). Our ConvNeXt results demonstrate that we can apply our approximations to real-world models, where we trade a minimal loss in accuracy (≤ 0.2 pp) for a significant gain in speedup.

We also evaluate how constant tuning affects end-to-end accuracy. Our compiler tunes constants holistically for the entire expression, rather than just the exponentiation is isolation. As Kadlec shows, foregoing holistic tuning has a profound impact on accuracy. We measured ConvNeXt Top-1 accuracy with two versions of our activations, one tuned holistically, and one where the exponentiation is tuned in

isolation. Performing the holistic tuning gained between 0.1 (Tiny) and 0.3 percentage points (Femto) of accuracy. Since this tuning is performed once during the deployment process, it introduces no model runtime overhead and can essentially be considered “free accuracy”.

6. Related Work

6.1. Bitcasting for Approximation

Using the bit-casting numbers to seed approximations has been common knowledge, especially in graphics-programming circles, and was most famously used in Quake III’s fast-inverse-square-root implementation (id Software, 1999). This approach of approximating the square root has been extensively studied in the literature since then (Lomont, 2003; Kadlec, 2010; Muller, 2020; Walczyk et al., 2021; Kusaka & Tanaka, 2022; Moroz et al., 2022).

Within machine learning, (Gustafsson & Hellman, 2021) explores a manual method to approximate arithmetic operations such as addition, multiplication, division, and taking the square root, skipping over primitives useful for machine learning. They do show this method working for other data types such as bfloat16, tf32, and DLfloat. (Schraudolph, 1999) presents a manual approach based on a C macro for performing exponentiation on doubles and floats that employs bit-casting techniques. There is work applying this technique to exponential linear units (ELUs) (Qiumei et al., 2019), but these approaches have not been generalized.

There is ongoing work in providing custom hardware for accelerating activation functions on the edge (Reggiani et al., 2025; Nilsson et al., 2014; Meher, 2010; Wuraola et al., 2021), trading of chip area for speed. However, this requires the production of custom chips, significantly impacting the cost of deployment. Directions here are split, with some work, such as (Reggiani et al., 2025), working with existing activations to allow the re-use of training infrastructure, and other work (Wuraola et al., 2021) providing a whole new family of activations which requires the development of new training infrastructure as well.

6.2. Compilers for Inference Acceleration.

There has been a surge in machine learning compilers and frameworks to accelerate neural network inference and training. Systems such as TVM (Chen et al., 2018) and its successors (Zheng et al., 2020; Feng et al., 2022; Bi et al., 2023) enable inference acceleration through automated schedule search and backend-specific code generation. In parallel, PyTorch 2.0 (Ansel et al., 2024) introduces `compile`, which leverages TorchInductor to lower high-level models into optimized kernels for efficient inference execution. Triton (Tillet et al., 2019) further exposes a compiler-based programming model for custom GPU kernels, enabling fine-

grained performance tuning at the operator level. Meanwhile, MLIR (Lattner et al., 2021) provides a multi-level intermediate representation that underpins many modern inference compilers, facilitating operator fusion and cross-layer optimization across abstraction boundaries. Complementary to these efforts, a line of fusion-based works (Zheng et al., 2022; Wu et al., 2025) focus on combining operators to reduce memory traffic and kernel launch overhead. These methods are orthogonal to the approach presented here because they focus on optimizing compute and memory patterns while preserving the operational semantics.

7. Conclusion

This paper introduces FFCC, a tool for generating accurate, fast and hardware-aware approximations for activation functions. The FFCC compiler uses bit-casting to efficiently and accurately approximate common machine-learning operations like log and exp, without the need for hardware-implemented fast-paths.

We evaluate the generated activation functions for accuracy, showing that they are within 0.019 of the completely accurate versions, which enables them to be used as drop-in replacements without incurring significant losses in model accuracy in most cases (between 0 to 0.2 percentage points). Micro-Benchmarks demonstrate order-of-magnitude speedups across architectures for our activations compared to exact implementations and even beat hand-tuned kernels from llama.cpp by 23×. Finally, we demonstrate that the results of these micro-benchmarks translate to real-world speedups on the ConvNeXt family of models.

FFCC thus enables the edge-deployment of activation functions that were previously deemed to exceed the compute-budgets of most edge devices, without requiring fundamental changes to model architectures or edge-hardware.

Impact Statement

This work represents an improvement to the kinds of machine-learning models that can be run on edge devices. This can enable more people to access models at lower costs, but since the methods presented in this paper are fundamentally model-agnostic, questions around responsible use remain with downstream adopters of these methods.

Acknowledgements

This work has received funding from the UKRI EPSRC Centre for Doctoral Training in Machine Learning Systems project ref. EP/Y03516X/1.

References

- Andri, R., Reggiani, E., and Cavigelli, L. Flex-SFU: Activation Function Acceleration With Nonuniform Piecewise Approximation. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 44(11):4236–4248, November 2025. ISSN 0278-0070, 1937-4151. doi: 10.1109/TCAD.2025.3558140. URL <https://ieeexplore.ieee.org/document/10949646/>.
- Ansel, J., Yang, E., He, H., Gimelshein, N., Jain, A., Voznesensky, M., Bao, B., Bell, P., Berard, D., Burovski, E., Chauhan, G., Chourdia, A., Constable, W., Desmairson, A., DeVito, Z., Ellison, E., Feng, W., Gong, J., Gschwind, M., Hirsh, B., Huang, S., Kalambarkar, K., Kirsch, L., Lazos, M., Lezcano, M., Liang, Y., Liang, J., Lu, Y., Luk, C. K., Maher, B., Pan, Y., Puhersch, C., Reso, M., Saroufim, M., Siraichi, M. Y., Suk, H., Zhang, S., Suo, M., Tillet, P., Zhao, X., Wang, E., Zhou, K., Zou, R., Wang, X., Mathews, A., Wen, W., Chanan, G., Wu, P., and Chintala, S. PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pp. 929–947, La Jolla CA USA, April 2024. ACM. ISBN 979-8-4007-0385-0. doi: 10.1145/3620665.3640366. URL <https://dl.acm.org/doi/10.1145/3620665.3640366>.
- Banbury, C., Reddi, V. J., Torelli, P., Holleman, J., Jeffries, N., Kiraly, C., Montino, P., Kanter, D., Ahmed, S., Pau, D., et al. Mlperf tiny benchmark. *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.
- Bi, J., Guo, Q., Li, X., Zhao, Y., Wen, Y., Guo, Y., Zhou, E., Hu, X., Du, Z., Li, L., Chen, H., and Chen, T. Heron: Automatically constrained high-performance library generation for deep learning accelerators. *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2023. URL <https://api.semanticscholar.org/CorpusID:257632862>.
- Bisbas, G., Lydike, A., Bauer, E., Brown, N., Fehr, M., Mitchell, L., Rodriguez-Canal, G., Jamieson, M., Kelly, P. H. J., Steuwer, M., and Grosser, T. A shared compilation stack for distributed-memory parallelism in stencil DSLs. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, pp. 38–56, La Jolla CA USA, April 2024. ACM. ISBN 979-8-4007-0386-7. doi: 10.1145/3620666.3651344. URL <https://dl.acm.org/doi/10.1145/3620666.3651344>.
- Biswas, K., Kumar, S., Banerjee, S., and Pandey, A. K. Smu: smooth activation function for deep networks using smoothing maximum technique. *arXiv preprint arXiv:2111.04682*, 2021.
- Chen, T., Moreau, T., Jiang, Z., Shen, H., Yan, E. Q., Wang, L., Hu, Y., Ceze, L., Guestrin, C., and Krishnamurthy, A. Tvm: An automated end-to-end optimizing compiler for deep learning. 2018. URL <https://api.semanticscholar.org/CorpusID:3296374>.
- DeVries, T. and Taylor, G. W. Improved Regularization of Convolutional Neural Networks with Cutout, November 2017. URL <http://arxiv.org/abs/1708.04552>. arXiv:1708.04552 [cs].
- Dosovitskiy, A. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.
- Elfwing, S., Uchibe, E., and Doya, K. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural networks*, 107:3–11, 2018.
- Fehr, M., Weber, M., Ulmann, C., Lopoukhine, A., Lücke, M. P., Degioanni, T., Vasiladiotis, C., Steuwer, M., and Grosser, T. xDSL: Sidekick Compilation for SSA-Based Compilers. In *Proceedings of the 23rd ACM/IEEE International Symposium on Code Generation and Optimization*, pp. 179–192, Las Vegas NV USA, March 2025. ACM. ISBN 979-8-4007-1275-3. doi: 10.1145/3696443.3708945. URL <https://dl.acm.org/doi/10.1145/3696443.3708945>.
- Feng, S., Hou, B., Jin, H., Lin, W., Shao, J., Lai, R., Ye, Z., Zheng, L., Yu, C. H., Yu, Y., and Chen, T. Tensorir: An abstraction for automatic tensorized program optimization. *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2022. URL <https://api.semanticscholar.org/CorpusID:250425865>.
- Gong, T., Zhu, L., Yu, F. R., and Tang, T. Edge intelligence in intelligent transportation systems: A survey. *IEEE Transactions on Intelligent Transportation Systems*, 24(9):8919–8944, 2023.
- Gustafsson, O. and Hellman, N. Approximate Floating-Point Operations with Integer Units by Processing in the Logarithmic Domain. In *2021 IEEE 28th Symposium on Computer Arithmetic (ARITH)*, pp. 45–52, Lyngby, Denmark, June 2021. IEEE. ISBN 978-1-6654-2293-2. doi: 10.1145/3620666.3651344.

- 10.1109/ARITH51176.2021.00019. URL <https://ieeexplore.ieee.org/document/9603384/>.
- He, K., Zhang, X., Ren, S., and Sun, J. Deep Residual Learning for Image Recognition, December 2015. URL <http://arxiv.org/abs/1512.03385>. arXiv:1512.03385 [cs].
- Hendrycks, D. and Gimpel, K. Gaussian Error Linear Units (GELUs), June 2023. URL <http://arxiv.org/abs/1606.08415>. arXiv:1606.08415 [cs].
- Howard, A., Sandler, M., Chu, G., Chen, L.-C., Chen, B., Tan, M., Wang, W., Zhu, Y., Pang, R., Vasudevan, V., et al. Searching for mobilenetv3. In *Proceedings of the IEEE/CVF international conference on computer vision*, pp. 1314–1324, 2019.
- Huang, G., Liu, Z., Maaten, L. v. d., and Weinberger, K. Q. Densely Connected Convolutional Networks, January 2018. URL <http://arxiv.org/abs/1608.06993>. arXiv:1608.06993 [cs].
- Iandola, F. N., Han, S., Moskewicz, M. W., Ashraf, K., Dally, W. J., and Keutzer, K. SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <0.5MB model size, November 2016. URL <http://arxiv.org/abs/1602.07360>. arXiv:1602.07360 [cs].
- id Software. Quake III Arena, 1999. URL <https://github.com/id-Software/Quake-III-Arena>. Source code released under GNU GPL v2.
- Kadlec, J. Improving the fast inverse square root, 2010. URL http://rrrola.wz.cz/inv_sqrt.html.
- Kusaka, T. and Tanaka, T. Fast and Accurate Approximation Methods for Trigonometric and Arctangent Calculations for Low-Performance Computers. *Electronics*, 11(15):2285, July 2022. ISSN 2079-9292. doi: 10.3390/electronics11152285. URL <https://www.mdpi.com/2079-9292/11/15/2285>.
- Lattner, C. and Adev, V. LLVM: A compilation framework for lifelong program analysis and transformation. pp. 75–88, San Jose, CA, USA, Mar 2004.
- Lattner, C., Amini, M., Bondhugula, U., Cohen, A., Davis, A., Pienaar, J. A., Riddle, R., Shpeisman, T., Vasilache, N., and Zinenko, O. Mlir: Scaling compiler infrastructure for domain specific computation. *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pp. 2–14, 2021. URL <https://api.semanticscholar.org/CorpusID:232127418>.
- Li, L., Ota, K., and Dong, M. Deep learning for smart industry: Efficient manufacture inspection system with fog computing. *IEEE Transactions on Industrial Informatics*, 14(10):4665–4673, 2018.
- Liang, S., Wu, H., Zhen, L., Hua, Q., Garg, S., Kaddoum, G., Hassan, M. M., and Yu, K. Edge yolo: Real-time intelligent object detection system based on edge-cloud co-operation in autonomous vehicles. *IEEE Transactions on Intelligent Transportation Systems*, 23(12):25345–25360, 2022.
- Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., Lin, S., and Guo, B. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF international conference on computer vision*, pp. 10012–10022, 2021.
- Liu, Z., Mao, H., Wu, C.-Y., Feichtenhofer, C., Darrell, T., and Xie, S. A ConvNet for the 2020s, March 2022. URL <http://arxiv.org/abs/2201.03545>. arXiv:2201.03545 [cs.CV].
- Lomont, C. Fast Inverse Square Root, February 2003. URL <https://lomont.org/papers/2003/InvSqrt.pdf>.
- Ma, N., Zhang, X., Zheng, H.-T., and Sun, J. ShuffleNet V2: Practical Guidelines for Efficient CNN Architecture Design, July 2018. URL <http://arxiv.org/abs/1807.11164>. arXiv:1807.11164 [cs].
- Meher, P. K. An optimized lookup-table for the evaluation of sigmoid function for artificial neural networks. In *2010 18th IEEE/IFIP International Conference on VLSI and System-on-Chip*, pp. 91–95. IEEE, 2010.
- Menshchikov, A., Shadrin, D., Prutyayov, V., Lopatkin, D., Sosnin, S., Tsykunov, E., Iakovlev, E., and Somov, A. Real-time detection of hogweed: Uav platform empowered by deep learning. *IEEE Transactions on Computers*, 70(8):1175–1188, 2021. doi: 10.1109/TC.2021.3059819.
- Mock, M., Edavettal, S., Langmead, C., and Russell, A. Ai can help to speed up drug discovery—but only if we give it the right data. *Nature*, 621(7979):467–470, 2023.
- Moon, J., Lee, H., Chu, J., Park, D., Hong, S., Seo, H., Jeong, D., Kong, S., and Ham, M. A new frontier of ai: On-device ai training and personalization. In *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Practice, ICSE-SEIP ’24*, pp. 323–333, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400705014. doi: 10.1145/3639477.3639716. URL <https://doi.org/10.1145/3639477.3639716>.

- Moroz, L., Samotyy, V., Kokosiński, Z., and Gepner, P. Simple multiple precision algorithms for exponential functions [tips & tricks]. *IEEE Signal Processing Magazine*, 39(4):130–137, 2022.
- Muller, J.-M. Elementary Functions and Approximate Computing. *Proceedings of the IEEE*, 108(12):2136–2149, December 2020. ISSN 0018-9219, 1558-2256. doi: 10.1109/JPROC.2020.2991885. URL <https://ieeexplore.ieee.org/document/9106347/>.
- Nilsson, P., Shaik, A. U. R., Gangarajaiah, R., and Hertz, E. Hardware implementation of the exponential function using taylor series. In *2014 NORCHIP*, pp. 1–4. IEEE, 2014.
- Perini, F. and Reitz, R. D. Fast approximations of exponential and logarithm functions combined with efficient storage/retrieval for combustion kinetics calculations. *Combustion and Flame*, 194:37–51, 2018.
- Qiumei, Z., Dan, T., and Fenghua, W. Improved convolutional neural network based on fast exponentially linear unit activation function. *IEEE Access*, 7:151359–151367, 2019. doi: 10.1109/ACCESS.2019.2948112.
- Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., and Sutskever, I. Robust speech recognition via large-scale weak supervision. In *International conference on machine learning*, pp. 28492–28518. PMLR, 2023.
- Ramachandran, P., Zoph, B., and Le, Q. V. Searching for Activation Functions, October 2017. URL <http://arxiv.org/abs/1710.05941>. arXiv:1710.05941 [cs].
- Reggiani, E., Andri, R., and Cavigelli, L. Flex-sfu: Accelerating dnn activation functions by non-uniform piecewise approximation. In *Proceedings of the 60th Annual ACM/IEEE Design Automation Conference, DAC '23*, pp. 1–6. IEEE Press, 2025. ISBN 9798350323481. doi: 10.1109/DAC56929.2023.10247855. URL <https://doi.org/10.1109/DAC56929.2023.10247855>.
- Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., and Chen, L.-C. MobileNetV2: Inverted Residuals and Linear Bottlenecks, March 2019. URL <http://arxiv.org/abs/1801.04381>. arXiv:1801.04381 [cs].
- Schraudolph, N. N. A fast, compact approximation of the exponential function. *Neural Computation*, 11(4):853–862, 1999.
- Seo, Y., Kim, J., and Park, U. Swish-t: Enhancing swish activation with tanh bias for improved neural network performance. *arXiv preprint arXiv:2407.01012*, 2024.
- Tillet, P., Kung, H.-T., and Cox, D. D. Triton: an intermediate language and compiler for tiled neural network computations. *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, 2019. URL <https://api.semanticscholar.org/CorpusID:184488182>.
- Tuli, S., Basumatary, N., Gill, S. S., Kahani, M., Arya, R. C., Wander, G. S., and Buyya, R. Healthfog: An ensemble deep learning based smart healthcare system for automatic diagnosis of heart diseases in integrated iot and fog computing environments. *Future Generation Computer Systems*, 104:187–200, 2020.
- Van Delm, J., Lydike, A., Dumoulin, J., Crols, J., Yi, X., Antonio, R., Woodruff, J., Grosser, T., and Verhelst, M. The Configuration Wall: Characterization and Elimination of Accelerator Configuration Overhead. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, pp. 265–280, Pittsburgh PA USA, March 2026. ACM. ISBN 979-8-4007-2165-6. doi: 10.1145/3760250.3762225. URL <https://dl.acm.org/doi/10.1145/3760250.3762225>.
- Walczyk, C. J., Moroz, L. V., and Cieśliński, J. L. Improving the Accuracy of the Fast Inverse Square Root by Modifying Newton–Raphson Corrections. *Entropy*, 23(1):86, January 2021. ISSN 1099-4300. doi: 10.3390/e23010086. URL <https://www.mdpi.com/1099-4300/23/1/86>.
- Wang, X., Tang, Z., Guo, J., Meng, T., Wang, C., Wang, T., and Jia, W. Empowering edge intelligence: A comprehensive survey on on-device ai models. *ACM Computing Surveys*, 57(9):1–39, 2025.
- Wu, M., Cheng, X., Liu, S., Shi, C., Ji, J., Ao, M. K., Velliengiri, P., Miao, X., Padon, O., and Jia, Z. Mirage: A {Multi-Level} superoptimizer for tensor programs. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*, pp. 21–38, 2025.
- Wulfert, L., Kühnel, J., Krupp, L., Viga, J., Wiede, C., Gembaczka, P., and Grabmaier, A. Aifex: A next-generation edge ai framework. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(6):4519–4533, 2024.
- Wuraola, A., Patel, N., and Nguang, S. K. Efficient activation functions for embedded inference engines. *Neurocomputing*, 442:73–88, 2021. ISSN 0925-2312. doi: <https://doi.org/10.1016/j.neucom.2021.02.030>. URL <https://www.sciencedirect.com/science/article/pii/S0925231221002769>.

Zhao, X., Ding, W., An, Y., Du, Y., Yu, T., Li, M., Tang, M., and Wang, J. Fast segment anything. *arXiv preprint arXiv:2306.12156*, 2023.

Zheng, L., Jia, C., Sun, M., Wu, Z., Yu, C. H., Haj-Ali, A., Wang, Y., Yang, J., Zhuo, D., Sen, K., Gonzalez, J. E., and Stoica, I. Ansor : Generating high-performance tensor programs for deep learning. In *USENIX Symposium on Operating Systems Design and Implementation*, 2020. URL <https://api.semanticscholar.org/CorpusID:219636278>.

Zheng, Z., Yang, X., Zhao, P., Long, G., Zhu, K., Zhu, F.,

Zhao, W., Liu, X., Yang, J., Zhai, J., Song, S. L., and Lin, W. Astitch: enabling a new multi-dimensional optimization space for memory-intensive ml training and inference on modern simt architectures. *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2022. URL <https://api.semanticscholar.org/CorpusID:247026617>.